

CESIZen

Plan de déploiement, maintenance et sécurisation

47

TESTS PASSÉS

v1.0.0

VERSION STABLE

CI/CD

GITHUB ACTIONS ✓

OWASP

7 RISQUES COUVERTS

Louis Beaujoin — CDA CESI Bloc 3 — Juillet 2026

Sommaire

1 Contexte & Enjeux

3 Environnements DEV / TEST / PROD

5 Pipeline CI/CD GitHub Actions

7 Sécurisation OWASP Top 10

9 Ticketing & SLA

11 Rollback & Résilience

13 Démonstration live

15 Conclusion

2 Architecture & Stack technique

4 Infrastructure Docker

6 Tests automatisés — 47 tests

8 Tests de sécurité automatisés

10 RGPD

12 Git Flow & Versioning

14 Métriques & Résultats



Le projet CESIZen

Plateforme web de bien-être mental pour un **Ministère fictif**. Développée en formation CDA CESI.

Fonctionnalités

- Exercices de respiration guidés (3 prédéfinis + personnalisable)
- Pages d'information santé mentale
- Espace personnel avec historique de sessions
- Back-office d'administration
- API REST avec authentification par token

Objectif Bloc 3

Déployer, maintenir et sécuriser une application web en conditions professionnelles :

- **Déploiement** : Docker · CI/CD · rollback
- **Sécurisation** : OWASP Top 10 · tests auto
- **Maintenance** : ticketing · SLA · Dependabot
- **Qualité** : 47 tests · Git Flow · SemVer

Résultat final

47/47 tests · CI/CD ☒ · Docker opérationnel · github.com/louisbeaujoin/cesizen



Couche	Technologie	Version	Rôle
Framework	Laravel	13	MVC, routing, ORM, auth, middleware
Langage	PHP	8.3	Backend · PHP-FPM en prod
CSS / Build	Tailwind CSS + Vite	4 / 7	Utilitaires CSS · Hot reload DEV · Bundle PROD
Base de données	MySQL	8.0	Données persistantes · migrations Eloquent
Tests	Pest	4.x	BDD-style · 47 tests · SQLite in-memory
Serveur DEV	Apache (Docker)	2.4	Port 8080 · mod_rewrite activé
CI/CD	GitHub Actions	—	Tests + Build Docker automatiques





Critère	DEV (WAMP)	TEST (Docker)	PROD (théorique)
Démarrage	<code>composer dev</code>	<code>docker compose up --build</code>	Pipeline CD
Port	8000	8080	443 (HTTPS)
Base de données	MySQL WAMP	MySQL 8 container	MySQL managé (RDS)
APP_DEBUG	true	false	false
Assets Vite	Hot reload	Compilés dans l'image	Compilés + CDN
Ressources min.	4 Go RAM	4 Go RAM · 3 Go disque	2 vCPU · 4 Go · 50 Go SSD
Rollback	—	Volume drop + rebuild	git revert + mysqldump

L'environnement TEST est celui réellement opérationnel dans ce projet. La production est décrite selon les standards professionnels.



2 services — docker-compose.yml

Service app

- Image : build local (PHP 8.3 Apache + Node 20)
- Port : **8080** → **80**
- Entrypoint : `entrypoint.sh`

Service db

- Image : `mysql:8.0`
- Port : non exposé (réseau interne)
- Healthcheck : évite que l'app démarre avant MySQL

Données démo : **admin@cesizen.fr** / password

entrypoint.sh — démarrage automatique

```
# .env absent de l'image → copier le modèle
[ ! -f .env ] && cp .env.example .env

# Générer la clé applicative
[ -z "$APP_KEY" ] && php artisan key:generate --force

# Attendre MySQL puis migrer
until php artisan migrate --force; do
    sleep 3
done

# Seeder si demandé
if [ "${DB_SEED:-false}" = "true" ]; then
    php artisan db:seed --force
fi

exec apache2-foreground
```



Job	Déclencheur	Étapes principales	Résultat
tests	Push main / develop	Setup PHP 8.3 · Composer · SQLite · php artisan test	47/47 tests ✓
docker	needs: tests seulement	docker build -t cesizen-app .	Image buildée ✓

Dernière exécution : ✓ success — 3 juillet 2026 — 1 min 44s

Si un test échoue → job docker annulé automatiquement. Zéro code cassé en TEST.



47

Tests passés

118

Assertions

2.5s

Durée

0

Échoués

Répartition des tests

Suite	Tests
Unit — Modèles	9
Feature — Auth	13
Feature — Respiration	12
Feature — Information	6
Feature — Sécurité	3
Exemples	4
TOTAL	47



Risque OWASP	Implémentation CESIZen	Statut
A01 — Contrôle d'accès	Middleware <code>auth</code> , rôles admin/user, routes protégées	✓
A02 — Cryptographie	Bcrypt 12 rounds, HTTPS prod, .env hors git	✓
A03 — Injection SQL	Eloquent ORM (requêtes préparées), validation Laravel	✓
A03 — XSS	Blade <code>{{ }}</code> (auto-escape), cast <code>(int)</code> paramètres JS	✓
A05 — Mauvaise config	APP_DEBUG=false en TEST/PROD, .env exclu du repo	✓
A07 — Auth défaillante	throttle:5,1 sur /connexion et /api/login → HTTP 429	✓
CSRF	Token @csrf sur tous les formulaires POST	✓



3 tests dans `tests/Feature/SecurityTest.php` prouvent que les mécanismes fonctionnent :

```
// Test 1 – 4 en-têtes HTTP de sécurité présents sur toutes les pages
test('security headers are present on home page', function () {
    $this->get('/')->assertHeader('X-Content-Type-Options', 'nosniff')
        ->assertHeader('X-Frame-Options', 'SAMEORIGIN');
});

// Test 2 – HTTP 429 après 5 tentatives de connexion
test('throttle middleware blocks after 5 login attempts', function () {
    for ($i = 0; $i < 5; $i++) { $this->post('/connexion', [...]); }
    $this->post('/connexion', [...])>assertStatus(429);
});

// Test 3 – XSS neutralisé par cast (int)
test('xss cast int protection on breathing page', function () {
    $this->get('/respiration?inspiration=5%3Balert(1)%2F%2F')
        ->assertStatus(200)->assertDontSee('alert(1)', false);
});
```

Vérification live confirmée

Throttle testé sur app Docker : HTTP 302×5 → HTTP 429 ✓ · En-têtes confirmés via Invoke-WebRequest ✓

GitHub Issues + Projects

Board Kanban — 4 colonnes

- Backlog → En cours → En revue → Terminé
- Templates pré-remplis (bug + feature)
- Labels de criticité colorés
- Fermeture automatique par commit ([Fixes #13](#))

Dependabot

- Composer + npm : vérification hebdo
- GitHub Actions : vérification mensuelle
- PRs automatiques + validation CI avant merge

SLA par criticité

Niveau	Prise en compte	Correction
Bloquant critique	< 1h	< 3h
Bloquant majeur	< 4h	< 24h
Mineur	< 48h	Sprint suivant

Exemple bloquant critique : erreur 500 sur la page de connexion → prise en compte immédiate, rollback si nécessaire, correctif en moins de 3h ouvrées.

Données collectées

- **Minimisation** : nom, email, mot de passe hashé uniquement
- Aucun tracking, aucune donnée de navigation
- Aucune donnée de santé réelle collectée

Durées de conservation

- Comptes inactifs > 3 ans → supprimés
- Sessions expirées > 90 jours → purgées
- Logs applicatifs : 30 jours

Droits utilisateurs

- **Accès** : profil visible dans l'espace personnel
- **Rectification** : modification nom, email, mot de passe
- **Effacement** : suppression de compte (admin)

En cas de violation

- Notification CNIL sous **72h**
- Notification utilisateurs concernés sous **72h**
- Ticket `bloquant-critique` ouvert immédiatement



Rollback code — Git

```
# Annuler le commit fautif (préserve l'historique)
git revert <hash>
git push origin main

# Revenir à un tag stable
git checkout v1.0.0
```

Rollback base de données

```
# Annuler la dernière migration
php artisan migrate:rollback

# Restaurer depuis backup
mysql -u root -p cesizen < backup.sql
```

Critères de déclenchement

Scénario	Délai
Erreur 500 en PROD	Immédiat
Régression majeure	< 30 min
Corruption données	Immédiat
Vulnérabilité critique	Immédiat + patch

Bonne pratique

mysqldump avant toute migration · Tags SemVer sur chaque release · git revert préféré au reset --hard



Architecture des branches

```
main ← PR seulement — tag v1.0.0
├──
├── develop ← intégration continue
├──
└── feature/* ← fonctionnalité isolée
```

Flux de travail

```
feature/* → PR → develop → PR → main → tag vX.Y.Z
```

Semantic Versioning

Format	Signification
v1.0.0	Première version stable
v1.1.0	Nouvelle fonctionnalité
v1.0.1	Correctif de bug
v2.0.0	Changement cassant

Conventional Commits

```
feat: nouvelle fonctionnalité
fix: correction de bug
docs: documentation
test: ajout de tests
chore: maintenance
```



A

Versioning + CI/CD
3 min

B

Ticketing GitHub
2 min

C

App Docker live
1 min

6

Minutes totales

A — GitHub Actions

- Historique des commits
- Dernier run CI 
- 2 jobs : tests → docker
- Branches main / develop

B — Ticketing

- Board Kanban 4 colonnes
- Template bug rempli
- Labels criticité
- PRs Dependabot

C — App live

- <http://localhost:8080>
- Connexion admin
- DevTools → en-têtes sécurité
- Exercice de respiration

Si problème

- `docker compose restart app`
- `docker compose down -v && up --build`
- Garder son calme !



47

Tests passés
(0 échoué)

118

Assertions
vérifiées

7

Risques OWASP
couverts

2

Jobs CI/CD
GitHub Actions

Critère	Objectif	Atteint ?
Tests automatisés	Suite complète passant en CI	✓ 47/47 tests
Sécurité	OWASP Top 10 adressé + tests auto	✓ 7 risques + 3 tests
Infrastructure	Docker reproductible	✓ up en 1 commande
CI/CD	Pipeline tests → build	✓ 2 jobs séquentiels
Ticketing	Système de suivi formalisé	✓ GitHub Issues + SLA
Documentation	README + dossier + rollback	✓ Complet
Versioning	Git Flow + SemVer + Conventional Commits	✓ v1.0.0

CESIZen — Bloc 3

Une application déployée, testée, sécurisée et maintenue avec une chaîne d'outils professionnelle cohérente.

- ✓ 47 tests automatisés · CI GitHub Actions · 2 jobs séquentiels
- ✓ Docker opérationnel · <http://localhost:8080> · données démo
- ✓ 7 risques OWASP · en-têtes · throttle · XSS · tests de sécurité
- ✓ Ticketing GitHub · SLA · Dependabot · rollback documenté
- ✓ Git Flow · SemVer v1.0.0 · Conventional Commits
- ✓ RGPD · minimisation · droits utilisateurs · notification CNIL

github.com/louisbeaujoin/cesizen · Louis Beaujoin · CDA CESI Bloc 3